

Oficina de Mineração de Repositórios de Software

PyDriller, análise de dados e topic modelling

Prof. João Paulo Biazotto

Universidade Estadual do Paraná – Campus Apucarana
Bacharelado em Ciência da Computação

Mineração de Repositórios de Software
2026

Objetivos da oficina

Ao final da oficina, espera-se que você consiga:

- explicar o que é Mineração de Repositórios de Software (MSR);
- identificar artefatos de software que podem ser minerados;
- transformar uma pergunta de Engenharia de Software em dados observáveis;
- coletar dados de commits com PyDriller;
- analisar dados com pandas;
- explorar mensagens de commit com topic modelling.

Estrutura da oficina

Organização sugerida

- 20–30 min: motivação e contextualização sobre MSR;
- 60 min: estudo de caso guiado;
- 80 min: desafio em grupos;
- 10 min: fechamento.

Ideia central

Não vamos apenas executar comandos. Vamos transformar rastros de desenvolvimento em evidências para responder perguntas de Engenharia de Software.

Roteiro dos slides

- 1 O que é MSR?
- 2 Por que MSR é importante?
- 3 Quais artefatos podem ser explorados?
- 4 Quais técnicas podem ser usadas?
- 5 Qual é o fluxo de um estudo MSR?
- 6 Como aplicar isso em um estudo de caso?

Motivação: software deixa rastros

Durante o desenvolvimento, equipes produzem muitos registros:

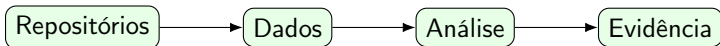
- commits;
- arquivos modificados;
- mensagens de commit;
- issues e bugs;
- pull requests;
- revisões de código;
- testes e integração contínua;
- discussões e documentação.

Pergunta

O que podemos aprender com esses rastros?

O que é MSR?

Conceito-chave: MSR é a área que extrai e analisa dados de repositórios de software para descobrir informações úteis sobre sistemas, processos e equipes.



Exemplo: usar commits para identificar arquivos que mudam com muita frequência.

MSR não é apenas mineração de código

MSR pode envolver diferentes dimensões do desenvolvimento:

- **Produto:** código, testes, arquitetura, documentação;
- **Processo:** frequência de mudanças, releases, integração contínua;
- **Pessoas:** autores, revisores, ownership, colaboração;
- **Problemas:** bugs, issues, mudanças corretivas, dívida técnica;
- **Texto:** mensagens, discussões, descrições de issues.

Por que MSR é importante?

- Permite estudar projetos reais em larga escala.
- Ajuda a entender como sistemas evoluem ao longo do tempo.
- Apoia decisões de manutenção com evidências históricas.
- Permite encontrar padrões difíceis de perceber manualmente.
- Conecta pesquisa empírica e problemas práticos de Engenharia de Software.

Exemplos de decisões apoiadas por MSR

- Quais arquivos merecem atenção em uma refatoração?
- Quais partes do sistema concentram correções?
- Quem conhece melhor determinado componente?
- O volume de manutenção aumentou nos últimos meses?
- Que tipos de mudança aparecem nas mensagens de commit?
- Quais módulos estão associados a maior risco de mudança?

Perguntas que podemos responder

Exemplos de perguntas de Engenharia de Software

- Onde o sistema muda mais?
- Quanto código é alterado em cada parte do projeto?
- Quais arquivos dependem de poucos desenvolvedores?
- Quais commits parecem estar relacionados a correções?
- Quais tópicos aparecem nas mensagens de commit?

Ponto importante

A pergunta vem antes da técnica.

Artefatos que podem ser explorados

Artefato	Exemplos de dados
Git	commits, autores, datas, arquivos, churn
Issues	bugs, features, labels, comentários
Pull requests	revisores, tempo, discussões, aprovação
Código	métricas, dependências, smells, testes
CI/CD	builds, falhas, tempo de execução
Discussões	e-mails, fóruns, chats, decisões

Artefato: histórico Git

O histórico Git permite observar:

- quem alterou;
- quando alterou;
- quais arquivos foram alterados;
- quantas linhas foram adicionadas/removidas;
- quais mensagens descrevem as mudanças.

Pergunta possível

Quais arquivos concentram maior volume de manutenção recente?

Exemplo de pesquisa: repositórios como fontes ativas

Ahmed Hassan argumenta que repositórios como controle de versão, bugs e logs podem deixar de ser apenas registros passivos e se tornar fontes ativas para apoiar decisões de manutenção.

Ideia para a prática

Se o histórico registra como o sistema mudou, podemos usá-lo para encontrar candidatos a hotspots de manutenção.

Hassan, A. E. (2008). The road ahead for Mining Software Repositories. FoSM. DOI:
10.1109/FOSM.2008.4659248.

Artefato: issues e bugs

Issues podem conter:

- título e descrição;
- labels;
- comentários;
- autor e responsáveis;
- tempo até resolução;
- links para commits ou pull requests.

Pergunta possível

Quem é o desenvolvedor mais indicado para corrigir um novo bug?

Anvik, Hiew e Murphy (2006). Who should fix this bug? ICSE. DOI: 10.1145/1134285.1134336.

Artefato: pull requests e code review

Pull requests permitem estudar:

- tempo até aceitação;
- número de revisores;
- quantidade de comentários;
- mudanças solicitadas;
- relação entre revisão e qualidade.

Pergunta possível

Mudanças maiores demoram mais para serem revisadas?

Artefato: integração contínua

Dados de CI/CD permitem investigar:

- frequência de builds;
- falhas recorrentes;
- tempo de execução;
- testes quebrados;
- etapas ignoradas ou com falhas permitidas.

Pergunta possível

Quais padrões de pipeline indicam fragilidade no processo de entrega?

Artefato: texto produzido pela equipe

Além do código, equipes produzem muito texto:

- mensagens de commit;
- descrições de issues;
- comentários em pull requests;
- documentação;
- discussões técnicas.

Na oficina

Vamos usar mensagens de commit para experimentar topic modelling.

Artefato → pergunta → técnica

Artefato	Pergunta	Técnica
Commits	Onde muda mais?	agregação
Commits	Quanto muda?	churn
Autores	Quem conhece o quê?	ownership
Mensagens	Que temas aparecem?	NLP / tópicos
Datas	Como evolui?	série temporal

RESEARCH ARTICLE OPEN ACCESS

Investigating Developer Sentiments in Software Components: An Exploratory Case Study of Gentoo

Tien Rahayu Tulili  | Ayushi Rastogi | Andrea Capiluppi 

Bernoulli Institute for Mathematics, Computer Science and Artificial Intelligence, University of Groningen, Groningen, the Netherlands

Correspondence: Tien Rahayu Tulili (t.r.tulili@rug.nl)

Received: 14 November 2024 | **Revised:** 13 March 2025 | **Accepted:** 28 March 2025

Funding: This work was supported by the Lembaga Pengelola Dana Pendidikan.

Keywords: collaborative software development | components | open-source software | sentiment analysis

Figura: Tulili et. al 2025

Exemplos de estudos com MSR

The Journal of Systems and Software 215 (2024) 112112



ELSEVIER

Contents lists available at [ScienceDirect](https://www.sciencedirect.com)

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss



Mining for cost awareness in the infrastructure as code artifacts of cloud-based applications: An exploratory study[☆]

Daniel Feitosa^{*}, Matei-Tudor Penca, Massimiliano Berardi, Rares-Dorian Boza, Vasilios Andrikopoulos

Bernoulli Institute for Mathematics, Computer Science and Artificial Intelligence, University of Groningen, The Netherlands

Figura: Feitosa et al. 2026

Investigating CI/CD-based Technical Debt Management in Open-source Projects

João Paulo Biazotto

Bernoulli Institute for Mathematics, Computer Science and
Artificial Intelligence
University of Groningen
Groningen, Groningen, Netherlands
Department of Computer Systems
University of São Paulo
São Carlos, São Paulo, Brazil
j.p.biazotto@rug.nl

Paris Avgeriou

Bernoulli Institute for Mathematics, Computer Science and
Artificial Intelligence
University of Groningen
Groningen, Groningen, Netherlands
p.avgeriou@rug.nl

Daniel Feitosa

Bernoulli Institute for Mathematics, Computer Science and
Artificial Intelligence
University of Groningen
Groningen, Groningen, Netherlands
d.feitosa@rug.nl

Elisa Yumi Nakagawa

Department of Computer Systems
University of São Paulo
São Carlos, São Paulo, Brazil
elisa@icmc.usp.br

Figura: Biazotto et al. 2026

Técnicas que podem ser usadas

- Estatística descritiva;
- visualização de dados;
- séries temporais;
- mineração de texto;
- topic modelling;
- aprendizado de máquina;
- análise de redes sociais;
- análise qualitativa complementar.

Técnica: estatística descritiva

- Contagens;
- médias e medianas;
- valores máximos e mínimos;
- distribuição dos dados;
- ranking de arquivos, autores ou períodos.

Na oficina

Vamos contar modificações por arquivo e autores por arquivo.

Técnica: séries temporais

Quando os dados possuem datas, podemos observar evolução:

- commits por mês;
- churn por período;
- autores ativos ao longo do tempo;
- mudanças antes/depois de uma release.

Pergunta possível

O projeto está ficando mais estável ou mais turbulento?

Técnica: visualização

Visualizações ajudam a encontrar padrões:

- barras para rankings;
- linhas para evolução temporal;
- dispersão para relacionar métricas;
- boxplots para distribuições.

Cuidado

Um gráfico não é uma conclusão. Ele é uma evidência a ser interpretada.

Técnica: mineração de texto

Texto aparece em muitos artefatos:

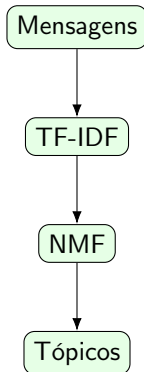
- mensagens de commit;
- issues;
- pull requests;
- documentação;
- comentários de revisão.

Na prática

Vamos transformar mensagens de commit em vetores numéricos usando TF-IDF.

Técnica: topic modelling

Topic modelling tenta encontrar grupos de palavras que aparecem juntas.



Importante

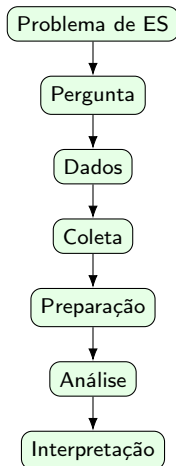
O algoritmo mostra palavras. A interpretação do tópico é humana.



Da pergunta aos dados

Pergunta	Dados necessários
Onde muda mais?	arquivo, commit
Onde muda mais código?	arquivo, linhas adicionadas/removidas
Quem conhece o arquivo?	arquivo, autor
Há sinais de correção?	mensagem, arquivo, commit
Quais temas aparecem?	mensagem de commit

Fluxo de um estudo MSR



Estudo de caso guiado

Cenário

Você entrou em uma equipe responsável pela manutenção de um projeto existente. A equipe suspeita que algumas partes do sistema concentram manutenção, mas ainda não possui evidências.

Objetivo

Usar o histórico Git para identificar candidatos a hotspots de manutenção e explorar os temas das mensagens de commit.

Projeto do estudo de caso

- Projeto: `psf/requests`;
- linguagem principal: Python;
- fonte de dados: histórico Git;
- janela: 500 commits mais recentes;
- unidade inicial de análise: arquivo modificado em commit.

Por que limitar a 500 commits?

Para manter a análise rápida e discutir a ideia de janela de observação.

Perguntas do estudo de caso

- **RQ1:** Quais arquivos foram modificados com maior frequência?
- **RQ2:** Quais arquivos apresentam maior churn de código?

Progressão

Onde muda? → Quanto muda? → Por que parece mudar?

Possibilidades de desafio

- ① Quais partes do projeto parecem mais instáveis?
- ② Onde existe concentração de conhecimento?
- ③ Como a manutenção mudou ao longo do tempo?
- ④ Onde estão concentradas as correções?

Resumo

MSR transforma rastros de desenvolvimento em evidências para apoiar perguntas de Engenharia de Software.

- Dados não falam sozinhos.
- A pergunta orienta a coleta e a análise.
- Métricas são aproximações, não verdades absolutas.
- Interpretação e limitações fazem parte do resultado.